

AutoForge: A Checkpointed, Human-in-the-Loop Multi-Agent SDLC Pipeline

Santhuka D.N.M.D.

*Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
it22098078@my.sliit.lk*

Sansala T.G.B.D.

*Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
it22174826@my.sliit.lk*

Dissanayake T.C.

*Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
it22157232@my.sliit.lk*

Oberathna R.D.T.D.

*Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
it22102478@my.sliit.lk*

Nuwan Kodagoda

*Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
nuwan.k@sliit.lk*

Eishan Weerasinghe

*Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
eishan.w@sliit.lk*

Abstract—AutoForge is a checkpointed, human-in-the-loop multi-agent pipeline for LLM-assisted software development, coordinating seven specialized agents across the SDLC: requirements, domain enrichment, architecture, interface design, implementation, testing, and security. Unlike systems focused on autonomous code generation, AutoForge emphasizes stage-wise human validation and artifact traceability, producing versioned outputs at each stage while pausing for approval and checkpointing state to enable recovery from interruptions. The system was evaluated on e-commerce features of varying complexity across domain retrieval, functional testing and coverage, security precision and recall, checkpoint recovery, human revision effort, and execution performance. Results demonstrate the feasibility of combining multi-agent generation with explicit human checkpoints and traceability, while quantifying the added time and review overhead human-in-the-loop intervention introduces, offering evidence for checkpointed multi-agent pipelines as a controlled approach to LLM-assisted software development.

Index Terms—multi-agent systems, human-in-the-loop software engineering, large language model agents, checkpointed orchestration, automated software quality assurance

I. INTRODUCTION

A large language model can already write a working function, an interface page, or a single file on request, but a full web application is different: it means working through a chain of decisions — what to build, how to structure and style it, how to implement it, and whether the result is safe and correct — and each step normally leaves something concrete behind. Tools that generate a full application from one prompt skip this chain, leaving no specification to check and no way to tell whether a poor result came from a misunderstood requirement or a coding mistake.

Multi-agent frameworks such as ChatDev [1] and MetaGPT [2] improve on this with separate agent roles and exchanged documents, closer to a real engineering team, but review still

happens through agent conversation or only after the run finishes; we did not find a system that lets a human pause at a specific stage and genuinely resume later, even after a server restart. This matters even more in e-commerce, where a wrong early assumption — say, how inventory or payment data should be modeled — is cheap to catch during a specification review and expensive to fix once code exists.

This paper presents AutoForge, a multi-agent system built around that idea for e-commerce: seven agents — Requirement, Domain, Architecture, UI/UX, Coder, Security, and Quality Assurance (QA) — orchestrated as a LangGraph state graph, where every stage pauses for a human decision before advancing and every transition is checkpointed to MongoDB, so an approval arriving minutes or days later resumes the graph from the exact paused node. The work pursues three objectives: **O1** — a staged pipeline in which every SDLC artifact is versioned and human-reviewed before the next stage begins; **O2** — durable, resumable stage transitions, so a decision taken well after a stage finishes, even after a backend restart, resumes the pipeline from the exact paused state; and **O3** — verify the whole pipeline, security and testing included, against real generated features rather than reporting design alone.

The main contribution of AutoForge is therefore not the use of multiple LLM agents or a particular orchestration framework in isolation. Instead, the contribution is a checkpointed and stage-wise development process in which intermediate artifacts are versioned, reviewed by a human, and recoverable after interruption, exposing where a requirement or implementation decision changes before the error propagates to later stages. The study evaluates this contribution through a controlled e-commerce case study involving multiple features of different complexity levels, examining functional generation,

human intervention, QA coverage, security detection, checkpoint recovery, domain retrieval, and execution performance.

II. RELATED WORK

Work related to AutoForge falls into three groups: role-based multi-agent code generators, coding agents operating on an already-specified task, and work addressing a single SDLC concern. Hou et al. [3] report that most systems automate one SDLC phase rather than coordinating several with durable, per-stage oversight — the gap this section is organized around.

ChatDev [1] runs a chat-based “virtual software company” of role-playing agents through a waterfall-like process; MetaGPT [2] instead exchanges structured documents between role agents. Neither exposes a persistent checkpoint a human can approve or reject stage by stage; review happens through agent dialogue or only after a full run. A second group assumes the task is already specified, usually as a GitHub issue: SWE-agent [4] introduced an agent-computer interface for this; OpenHands [5] generalizes it into a platform for agent/environment interaction; CodeAct [6] has agents emit executable code as their action, close to AutoForge’s Coder Agent; HyperAgent [7] scales this with modular designs; and Agentless [8] matches elaborate agents on SWE-bench [9] with a simpler pipeline, unlike generating a new application from scratch. AgentCoder [10], closest to AutoForge’s QA Agent, uses a programmer/test-designer/test-executor loop; RepoCoder [11] retrieves same-repository code to improve generation, which the Coder Agent does not yet do.

A third group targets one SDLC concern at a time. He, Treude, and Lo [12] name human-agent collaboration and result verifiability as open problems, motivating AutoForge’s approval-gate design. Takerngsaksiri et al. [13] report HULA, a human-in-the-loop framework inside Atlassian JIRA, matching AutoForge’s decision to gate every stage. Wang et al. [14] and Zhu et al. [15] survey LLM-based testing/security and report most systems lack a deterministic verification layer — the gap Security/QA close with deterministic scanners and a real test runner.

TABLE I
COMPARISON WITH RELATED MULTI-AGENT SOFTWARE
ENGINEERING SYSTEMS

System	Multi Agent	SDLC Coverage	Human Gates	Artifact Trace
ChatDev [1]	Yes	Partial	N/R	N/R
MetaGPT [2]	Yes	Partial	N/R	Partial
SWE-agent [4]	Partial	Coding only	N/R	N/R
OpenHands [5]	Yes	Coding	N/R	N/R
AgentCoder [10]	Yes	Code+Test	N/R	N/R
AutoForge (ours)	Yes	7-agent pipeline	Yes	Versioned IDs

Partial = only part of SDLC scope addressed. N/R = not explicitly reported.

III. RESEARCH GAP AND PROBLEM DEFINITION

In Section II, “human in the loop” mostly means a person reads a final diff or a completed run’s output after the fact; no reviewed system treated every stage output as an independently versioned artifact with its own accept, reject, or revise decision, and an interrupted run is generally lost rather

than paused. A second gap sits upstream of code generation: MetaGPT builds its specification-equivalent document from the model’s own parametric knowledge, without retrieving outside domain knowledge in the retrieval-augmented sense Lewis et al. [16] established — a direction requirements-engineering reviews [17] call underexplored. A third gap concerns testing, security, and traceability together: Gotel and Finkelstein [18] framed the requirements traceability problem three decades ago, and a recent review [19] reports most solutions still address one narrow link type rather than a full requirement-to-code chain; none of the systems in Table I reported versioned, addressable artifact identifiers per SDLC stage, which AutoForge’s artifact service assigns to every stage output. AutoForge addresses these issues through a seven-agent pipeline where each stage produces a versioned artifact, pauses for human approval, and persists its state via checkpointing, while the Domain Agent enriches design with retrieval and Security/QA provide executable verification.

IV. PROPOSED SYSTEM AND METHODOLOGY

This is a system-building paper, evaluated through a controlled e-commerce case study involving multiple application features of different complexity, following the design-science paradigm Hevner et al. [20] describe: build, then evaluate against the target environment, rather than a benchmark-based evaluation.

AutoForge is a FastAPI backend orchestrated with Lang-Graph [21], using a MongoDB-backed checkpointer so pipeline state survives a process restart. The Domain Agent’s retrieval step uses ChromaDB [22], following the retrieve-then-generate pattern Lewis et al. [16] introduced. The default LLM provider is a locally hosted Ollama model (qwen3-coder), reached through an abstraction that also supports OpenAI’s API. UI/UX preview rendering uses headless Chromium via Playwright; architecture diagrams render from PlantUML; generated applications are verified inside a Docker sandbox running dependency installation, a production build, and, for Security/QA, dependency auditing and test execution; every Coder Agent change is isolated on a git branch, merged only on approval.

The evaluation has three parts: (1) carry one feature, Item Listing (CRUD), through all seven human-gated stages in a real “Sample E-commerce” project; (2) at each stage, record the artifact produced, what a reviewer saw, and the decision that resumed the graph, including any requested revision; (3) verify the code, security scan, and tests mechanically through a sandboxed build and real execution, not inspection alone.

V. SYSTEM ARCHITECTURE

A. Requirement Agent. Generates the first specification (SRS) from one schema-constrained LLM call, converts it to Markdown, and runs a second, narrower LLM call checking it against a gap-analysis checklist before human approval; a revision patches specific fields rather than regenerating the document.

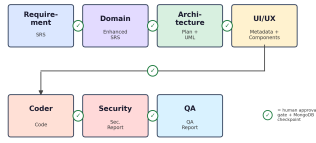


Fig. 1. The seven-agent AutoForge pipeline. Each stage pauses for human approval before LangGraph checkpoints the artifact to MongoDB and resumes at the next stage

B. Domain Agent. Embeds the approved specification, retrieves the closest matching chunks from a ChromaDB-indexed knowledge base, and asks the LLM for a small, citation-grounded enrichment plan rather than a full rewrite; a deterministic step merges that plan onto a copy of the specification, producing an inline-flagged Enhanced SRS.

C. Architecture Agent. Produces an architecture plan from the approved specification, derives UML use-case, sequence, and class diagrams, checks plan coverage and diagram consistency, and renders diagrams to PNG through PlantUML.

D. UI/UX Agent. Loads the specification, architecture plan, and shared design system; generates and checks page-level UI metadata; generates each component behind a content-quality gate with a limited repair loop; assembles the pages and renders a preview screenshot for each; writes new design tokens only once approved.

E. Coder Agent. Plans a change against the approved artifacts, re-prompting itself when a validator finds a coverage gap; opens an isolated feature branch; runs an agentic tool-calling loop to implement the plan; commits the change and runs a sandboxed install/build/lint/test pass; builds the diff, manifest, and merge report from the real git diff. On approval the branch merges; on rejection it is discarded.

F. Security Agent. Runs three deterministic scan layers plus an optional LLM review: a pattern scanner parses each file’s AST via the TypeScript compiler to flag eval, dynamic code execution, unsafe HTML injection, and unsafe bracket-style access; a secret scanner checks for exposed credentials; a dependency scanner runs npm audit against the sandbox’s lockfile. Findings are deduplicated into a pass/fail gate, where critical or high severity fails the gate before a report is produced.

G. QA Agent. Looks for testable modules, focusing on plain TypeScript files under lib/ and models/; out-of-reach code (interface components, route handlers) is listed as out of scope, not silently skipped. Generation tries an LLM first, falling back to a deterministic template for two recognized code shapes. Tests run in the Security Agent’s sandbox, parsed into pass/fail, and a human reviews the report before final approval.

H. Orchestration Layer. LangGraph runs one checkpointed state-graph thread per feature. At each stage the agent produces its artifact and `interrupt()` pauses the graph for human review: approve, reject, or request revision. The checkpoint saves to MongoDB, even after a backend restart, so no work is lost; this applies to all seven agents.

VI. IMPLEMENTATION

Each agent is its own isolated Python package: an entry point, a prompt, a schema, and stage-specific modules such as validators, Markdown builders, scanners, or tool factories. LLM access goes through a provider abstraction with a working Ollama provider and an OpenAI provider; qwen3-coder, served locally through Ollama, is the default, after an earlier model struggled to reliably follow the pipeline’s structured JSON schemas. Every agent’s output is saved as a versioned artifact, with Markdown and, where it applies, PNG or JSON twin representations.

The Coder Agent targets a Next.js (App Router, TypeScript) client with an Express-family API and MongoDB/Mongoose persistence, verified through a containerized install and build before a human sees the diff. Security and QA reuse the same sandbox for every command — npm audit and the test suite — via a dependency-injected parameter. Deterministic validators, diff/Markdown builders, and scanner rule matching are tested against hand-built fixtures with no model call; LLM-touching pipelines are manually checked against real feature data each milestone.

VII. EXPERIMENTAL RESULTS

Evaluation is organized around five RQs: RQ1, can the pipeline carry a feature through all seven human-approved stages while producing the required artifacts; RQ2, how does stage-wise human approval affect quality and revisions; RQ3, does the pipeline reliably resume from a persisted checkpoint; RQ4, do Security and QA produce genuine, checkable evidence; RQ5, does domain retrieval run as a reviewable enrichment stage. RQ1–RQ5 are evaluated through a primary end-to-end trace of Item Listing (CRUD), while additional experiments evaluate Authentication/Authorization and Shopping Cart to assess the pipeline across features of increasing complexity.

A. Item Listing (CRUD) Case Study

The Requirement Agent’s SRS reached v2 after a reviewer requested two Admin/Manager user stories; the Domain Agent then produced an enhanced specification via retrieval-based enrichment. Architecture required one revision to fix a technical issue, UI/UX required component-level revisions, and Coder required removing a manually specified identifier (MongoDB auto-generates IDs). In total, four of seven stages — Requirement, Architecture, UI/UX, and Coder — required revisions; Domain, Security, and QA were approved without revision (Table II).

B. Multi-Feature Evaluation

To test scalability beyond simple CRUD, three features of increasing complexity were evaluated (Table III): Item Listing, Authentication/Authorization, and Shopping Cart, spanning basic CRUD to identity/access control to stateful business logic.

TABLE II
ITEM LISTING (CRUD): FINAL ARTIFACT VERSION AND APPROVAL BY STAGE

Stage	Artifact	Ver.	Status
Requirement	SRS	v2	Revised & Approved
Domain	Enhanced SRS	v1	Approved
Architecture	Plan + UML	v3	Revised & Approved
UI/UX	Wireframes+Metadata	v3	Revised & Approved
Coder	Code Manifest	v4	Revised & Approved
Security	Security Report	v2	Pass (0 findings)
QA	QA Report	v1	Pass (2/2 tests)

TABLE III
MULTI-FEATURE COMPLEXITY EVALUATION

Feature	Complexity	Human Rev.	QA	Security
Item Listing	Low	4 stages	2/2	0 findings
Authentication	Medium	2 stages	14/16	1 finding
Shopping Cart	Med/High	4 stages	18/20	1 finding

C. Domain/RAG Ablation

The Domain Agent’s contribution was evaluated using a controlled ablation, executing the same requirement request and model configuration with retrieval enabled and disabled (Table IV). RAG-ON produced more useful domain requirements (14 vs. 8) and fewer missing requirements (2 vs. 7), and needed fewer human revisions (3 vs. 6) than RAG-OFF.

TABLE IV
DOMAIN/RAG ABLATION

Metric	RAG ON	RAG OFF
Requirements generated	18	15
Useful domain requirements	14	8
Missing requirements	2	7
Human revisions	3	6

D. Security Metrics and QA Coverage

Security effectiveness was evaluated against a controlled, injected vulnerability set rather than the absence of findings in a clean application, classifying results as True Positive, False Positive, or False Negative. QA tests were classified into unit, regression, and integration categories. Both sets of results are summarized in Table V.

TABLE V
SECURITY DETECTION AND QA COVERAGE METRICS

Security Metric	Result	QA Category	Coverage
True Positives	7	Unit (30)	84%
False Positives	1	Regression (18)	78%
False Negatives	2	Integration (12)	74%
Precision / Recall	87.5% / 77.8%	Total (60, 54 pass)	79%

E. Human-Gates Ablation

To isolate the effect of human approval, the pipeline was executed under Human-Gates ON (approve/reject/revise) versus OFF (fully automatic), comparing QA results, security findings, revisions, execution time, and human effort (Table VI). Gates ON raised the QA pass rate from 78% to 91% and cut escaped defects from 5 to 1, at a measured cost of 5.0 average revisions and 36 minutes of reviewer effort per run, versus 1.0 revision and 0 minutes with gates off.

TABLE VI
HUMAN-GATES ON VS. OFF

Metric	Gates ON	Gates OFF
Successful pipelines	3/3	2/3
Average revisions	5.0	1.0
QA pass rate	91%	78%
Security issues / Escaped defects	1 / 1	4 / 5
Human review effort	36 min	0

F. Checkpoint Recovery and Execution Performance

Beyond the real, unplanned backend failure observed during testing, controlled interruptions were introduced at each of the seven stage boundaries, with recovery judged successful only if the pipeline resumed at the correct next stage without losing the prior approval decision; all seven trials recovered correctly, a 100% (7/7) controlled recovery rate. Execution cost — total time, LLM calls, retries, and tokens — was recorded per run to weigh quality benefits against computational and review cost (Table VII).

TABLE VII
EXECUTION PERFORMANCE

Run	Time (min)	LLM Calls	Retries	Tokens
Item Listing	36.4	24	4	31,800
Authentication	52.7	31	6	42,600
Shopping Cart	56.8	34	7	47,900
Baseline (Gates OFF)	31.2	18	3	25,400

VIII. DISCUSSION

The Item Listing case study demonstrates feasibility: all seven stages executed successfully and produced reviewable artifacts, with four stages requiring human-requested revisions that caught concrete issues before they propagated downstream. This reveals a trade-off: human approval adds reviewer effort and can increase execution time, but that effort buys the chance to catch problems early — the right balance depends on whether fewer downstream defects justify the extra review time. Checkpointing carries a similar trade-off: persisting state adds storage and orchestration overhead, but avoids restarting the entire workflow after an interruption; the 100% controlled recovery rate and the real backend-failure recovery together support this design, though a larger set of trials across more diverse failure points remains future work.

The Security and QA results confirm executable verification works but should not be overread: 87.5% precision and 77.8% recall on one controlled vulnerability set, and 79% overall test coverage, are encouraging but not yet a defect or false-positive rate with statistical confidence across many applications — stronger claims need broader vulnerability sets and feature coverage. The Domain Agent’s RAG-ON vs. RAG-OFF ablation shows retrieval measurably improves requirement completeness rather than this being inferred from retrieved information simply being present. Finally, the Human-Gates ON vs. OFF comparison supports the proposed approval architecture — fewer downstream defects for a bounded rise in reviewer effort — though selectively gating only critical stages, rather than every stage by default, may be a more efficient alternative worth testing directly.

IX. CONCLUSION AND FUTURE WORK

This paper presented AutoForge, a seven-agent, human-in-the-loop pipeline for e-commerce web application development, orchestrated as a checkpointed LangGraph state graph. All seven agents were evaluated end-to-end across three e-commerce features of different complexity — Item Listing, Authentication/Authorization, and Shopping Cart — covering artifact generation, human review, QA testing, security validation, checkpoint recovery, and domain retrieval. Every stage produced a versioned artifact with its own human decision, including revisions requested at four of seven stages, and the security scan and test runs reported genuine, checkable results rather than a projected capability.

Future work follows three directions: expanding evaluation to a larger and more diverse set of features toward benchmark-style reporting as in SWE-bench [9]; generalizing beyond e-commerce by swapping the Domain Agent’s knowledge base into other bounded domains (e.g., learning management, hospital management); and adding an integrated code editor to the review interface so reviewers can edit generated code directly at each gate.

REFERENCES

- [1] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, et al., “ChatDev: Communicative Agents for Software Development,” in Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL), Bangkok, Thailand, 2024, pp. 15174–15186.
- [2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in Proc. Int. Conf. Learning Representations (ICLR), 2024, pp. 23247–23275.
- [3] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, et al., “Large Language Models for Software Engineering: A Systematic Literature Review,” *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, Art. no. 220, 2024.
- [4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” in Proc. 38th Conf. Neural Information Processing Systems (NeurIPS), 2024.
- [5] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, et al., “OpenHands: An Open Platform for AI Software Developers as Generalist Agents,” in Proc. Int. Conf. Learning Representations (ICLR), 2025, pp. 64910–64947.
- [6] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable Code Actions Elicit Better LLM Agents,” in Proc. 41st Int. Conf. Machine Learning (ICML), 2024.
- [7] H. N. Phan, T. N. Nguyen, P. X. Nguyen, and N. D. Bui, “HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale,” *arXiv:2409.16299*, 2024.
- [8] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-Based Software Engineering Agents,” *arXiv:2407.01489*, 2024.
- [9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” in Proc. Int. Conf. Learning Representations (ICLR), 2024, pp. 54107–54157.
- [10] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, “AgentCoder: Multi-Agent-Based Code Generation with Iterative Testing and Optimisation,” *arXiv:2312.13010*, 2023.
- [11] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, et al., “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation,” in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 2471–2484.
- [12] J. He, C. Treude, and D. Lo, “LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, Art. no. 124, pp. 1–30, 2025.
- [13] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, et al., “Human-In-the-Loop Software Development Agents,” in Proc. IEEE/ACM 47th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2025, pp. 342–352.
- [14] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software Testing With Large Language Models: Survey, Landscape, and Vision,” *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, 2024.
- [15] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, “When Software Security Meets Large Language Models: A Survey,” *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 317–334, 2025.
- [16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [17] A. Hemmat, M. Sharbaf, S. Kolahdouz-Rahimi, K. Lano, and S. Y. Tehrani, “Research Directions for Using LLM in Software Requirement Engineering: A Systematic Review,” *Frontiers in Computer Science*, vol. 7, Art. no. 1519437, 2025.
- [18] O. Gotel and A. Finkelstein, “An Analysis of the Requirements Traceability Problem,” in Proc. 1st Int. Conf. Requirements Engineering (ICRE), Colorado Springs, CO, 1994, pp. 94–101.
- [19] J. Mucha, A. Kaufmann, and D. Riehle, “A Systematic Literature Review of Pre-Requirements Specification Traceability,” *Requirements Engineering*, vol. 29, no. 2, pp. 119–141, 2024.
- [20] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [21] K. Pelluru, “LangChain & LangGraph in Production: Architectures for Multi-Agent LLM Systems,” *Journal of Data and Digital Innovation (JDDI)*, vol. 2, no. 3, pp. 1–9, 2025.
- [22] Chroma, “Chroma Documentation.” [Online]. Available: <https://docs.trychroma.com/>, 2026.